2520030117

SKILL – 16

Background Jobs and Job Management

Aim

To detect background jobs, launch non-blocking processes, return the prompt immediately, store job information, monitor execution, and test background tasks.

Objective

- To detect background jobs.
- To launch non-blocking processes.
- To return the prompt immediately.
- To store job information.
- To monitor process execution.
- To test background tasks.
- To create a job table.
- To store process groups.
- To track job states.
- To update job status.
- To remove completed jobs.
- To validate job records.

Problem Statement

To implement a shell mechanism that can identify background commands, execute them without blocking the shell, immediately return the prompt, and maintain information about running jobs and their states.

Theory

1. Background Jobs
Background jobs allow a command to run without blocking the shell.

2. Non-Blocking Process
The shell starts the process and immediately returns the prompt to the user.

3. Job Information
Information about each background process is stored so that its execution can be monitored.

4. Job Table
A job table stores details such as process ID, process group, and current job state.

5. Job States

The shell tracks whether a job is running, completed, or has another status.

Algorithm

1. Start the shell.
2. Read the command from the user.
3. Check whether the command is a background command.
4. Create a child process.
5. Execute the command in the child process.
6. Store the process information in the job table.
7. Return the prompt immediately.
8. Monitor the background process.
9. Update the job status.
10. Remove completed jobs.
11. Validate the job records.
12. Continue accepting commands.

Data Used

Commands, process IDs, process groups, job states, and job records are used to manage background processes.

Operations Used

- `fork()`
- `execvp()`
- `waitpid()`
- Process ID management
- Job table management
- Process status tracking

Job Management Structure

Command
↓
Background Job
↓
Child Process
↓
Job Table
↓
Track State
↓
Update / Remove Job

Testing

- Test a background command.

- Verify that the prompt returns immediately.
- Check that the job is stored in the job table.
- Monitor the running job.
- Verify the job status after completion.
- Remove completed jobs.
- Validate the stored job information.

Observation

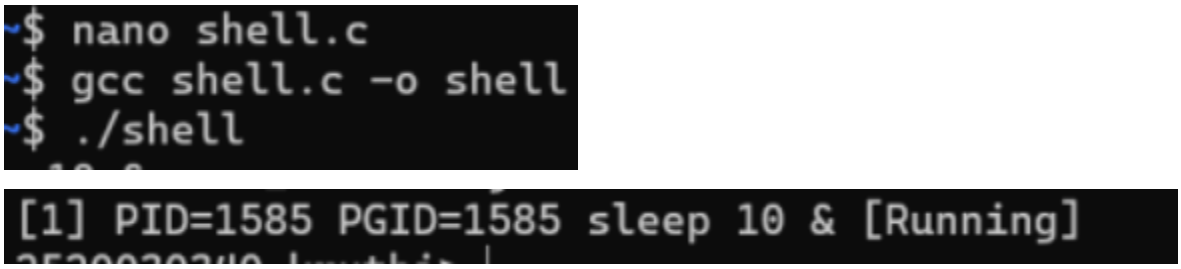
- Background processes execute without blocking the shell.
- The prompt returns immediately.
- Job information is stored correctly.
- Process states are tracked and updated.
- Completed jobs are removed from the job table.

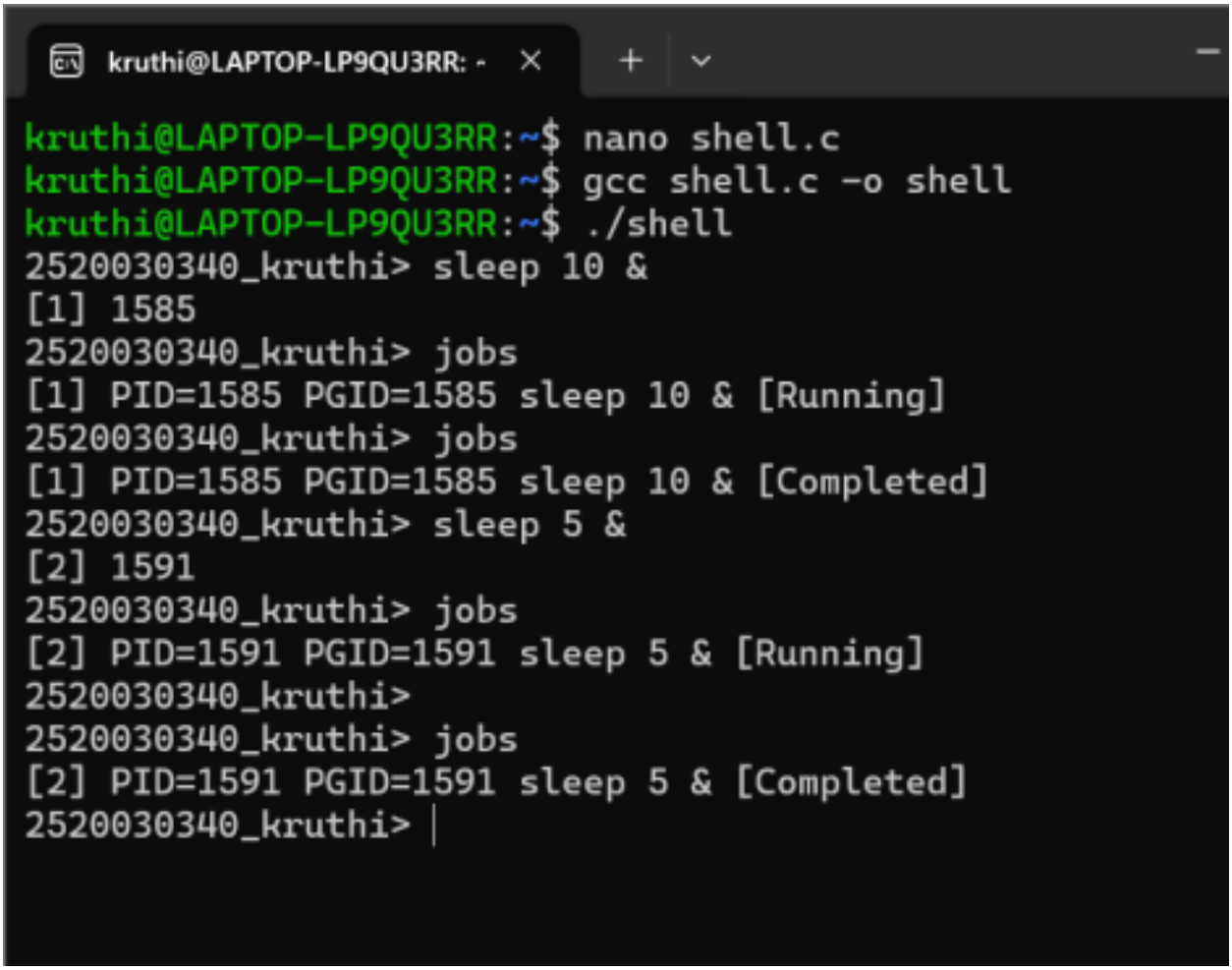
Result

Background job execution and job management were successfully implemented and tested.

Conclusion

The practical demonstrates how a shell can execute processes in the background, immediately return control to the user, and maintain a job table to monitor and manage background





Sample Output

kruthi@LAPTOP-LP9QU3RR:~$ ./shell

25200303117_vaishali>> sleep 10 &
[1] 1585
25200303117_vaishali> jobs
[1] PID=1585 PGID=1585 sleep 10 & [Running]

25200303117_vaishali>> jobs
[1] PID=1585 PGID=1585 sleep 10 & [Completed]

25200303117_vaishali>> sleep 5 &
[2] 1591
25200303117_vaishali> jobs
[2] PID=1591 PGID=1591 sleep 5 & [Running]

25200303117_vaishali> jobs
[2] PID=1591 PGID=1591 sleep 5 & [Completed]

Output Screenshots

Screenshot 1 – Background Job Running

Your first screenshot shows:

sleep 10 &
[1] 1585
jobs
[1] PID=1585 PGID=1585 sleep 10 & [Running]

Screenshot 2 – Background Job Completed

Your second screenshot shows:

[1] PID=1585 PGID=1585 sleep 10 & [Completed]

Screenshot 3 – Multiple Background Jobs

Your third screenshot shows the second job:

sleep 5 &
[2] 1591
jobs
[2] PID=1591 PGID=1591 sleep 5 & [Running]

jobs
[2] PID=1591 PGID=1591 sleep 5 & [Completed]

Testing

Test Case	Input	Expected Result
1	`sleep 10 &`	Background job starts and prompt returns immediately
2	`jobs`	Running job information is displayed 3 Wait for job completion Job status changes to Completed
4	`sleep 5 &`	Second background job is created 5 `jobs` PID, PGID and job status are displayed

Observation

- Background processes were launched successfully.
- The shell returned the prompt without waiting for the background process.
- PID and PGID were stored in the job table.
- Running and completed states were tracked correctly.
- Multiple background jobs were handled successfully.

Result

The background job management functionality was successfully implemented. The shell detected background commands, launched non-blocking processes, stored job information, monitored their states, and updated the job status after completion.

Conclusion

The experiment successfully demonstrated background process management in a custom shell using process IDs, process groups, job tables, and process-status tracking.